Brady's Squad: Human-led AI teams

Install it, run it, read what it wrote

Session M5-01 | 3 hours

Agenda

Segment	Time
What Squad is	10 min
Charters, routing, shared memory	15 min
How it runs, and Ralph	10 min
Demo: a live run	15 min
Team or one agent?	7 min
Lab handoff	3 min

What Squad is

An **open-source tool** by Brady Gaster. Not a GitHub product.

- A team of AI specialists that **live in your repo as files**
- You talk to one **coordinator**
- It starts specialists in parallel, each with its own context
- A **Scribe** records decisions. **Ralph** watches the work queue
- You review and merge. The human stays accountable

The team

Role	Job
Coordinator	Reads your request, routes it, starts agents
Lead	Triages, reviews, unblocks
Developers, tester	Do the work in their area
Scribe	Merges decisions into one file
Ralph	Monitors issues, PRs, and CI

Roles are **proposed from your repo**. You accept or change them.

What gets created

```
.squad/
├── team.md          # roster
├── routing.md       # who handles what
├── decisions.md     # shared rules, read by all
├── agents/<name>/
│   ├── charter.md   # job description
│   └── history.md   # what this agent learned
├── casting/         # persistent names
└── orchestration-log/
```

Commit it. Anyone who clones gets the team.

Check the Squad README for your version. File names move.

Charters and routing

Charter: what an agent owns, what it doesn't handle, how it works.

Routing: a table the coordinator reads first.

Pattern	Owner
`src/**`	Developer
`tests/**`	Tester
`docs/**`	Writer

Order: **named request**, then **path rule**, then **skill match**. No match? The lead triages.

Shared and personal memory

	Shared	Personal
File	`decisions.md`	`agents/<name>/history.md`
Read by	Every agent	That agent
Holds	Team rules and choices	Project knowledge for one role

Say "**always**" or "**never**" and it becomes a team rule.

It is Markdown in git. You can read it, diff it, and fix it.

Product format or your choice?

Squad defines: the folder layout, roster, routing, charters, decisions.

Your run chooses: agent names, routing rows, decision wording, error formats.

A sample `.squad/` file is an example. **Don't copy it as a requirement.**

How it runs

```
npm install -g @bradygaster/squad-cli@latest
squad init
copilot --agent squad
```

- **Copilot CLI** is the main interface. VS Code: pick the **Squad** agent.
- `squad` is for setup and checks: `init`, `doctor`, `upgrade`, `watch`
- The coordinator checks dependencies, runs independent work **in parallel**, dependent work in order

Ralph, the work monitor

Form	What it does
In a chat session	Checks GitHub for more work after each batch
`squad watch`	Polls on a timer. Triage only, or `--execute`
GitHub Actions heartbeat	Triages labeled issues between sessions

Needs `gh` access. Triages from **`routing.md`**.

Weak routing means weak triage.

Demo

1. `squad init`
2. `copilot --agent squad`, describe the project
3. Edit the proposed roster, accept
4. Give a multi-part task
5. Watch who starts, who waits
6. Review. Add a directive
7. Read `decisions.md` and `orchestration-log/`

A team can help

- Parts are **independent**: docs, tests, separate modules
- Roles need **different context**
- Work spans **many sessions** and needs memory
- A backlog needs **triage**

A team costs

- **Tokens.** Each agent loads its own context, plus the coordinator.
- **Overhead.** Splitting and collecting takes time. Small tasks get slower.
- **Conflicting edits.** Two agents, one file.
- **Review load.** More output, plus memory files.
- **Memory drift.** Stale decisions mislead every agent.

Start with one agent. Add a team when you hit a ceiling you can name.

Common mistakes

- Using a team for a one-minute task
- Splitting work that edits the same file
- `--yolo` on a repo you care about
- Never reading `decisions.md`
- Ralph without `gh` access or a good routing table
- Trusting docs over your installed version

Lab handoff

Part	Work
1 to 2	Install Squad, create the team
3 to 4	Give a four-part task, review, steer
5	Read `.squad/`
6	Same task with one agent. Compare
7	Ralph on real issues

Use the sample project and a **throwaway repo** for Ralph.